Forest Link Protocol: A Multi-Protocol Adaptive Mesh Network for Reliable Data Transfer in Dense Forest Environments

Po Haoting*, Ong Tun Siang*, Kenny Leck*, Chia Wei Sheng*
*Singapore Institute of Technology
CSC2106: IoT Protocols and Networks, Group GP01
{2401280, 2402091, 2403543, 2400953}@sit.singaporetech.edu.sg

***Abstract*—This paper presents the design and implementation of Forest Link Protocol (FLP), a custom application-layer mesh protocol for IoT sensor deployments in environments where internet connectivity is intermittent and signal attenuation is high. Unlike standard IoT solutions that rely on continuous cloud connectivity or specialized LPWAN hardware, FLP utilizes commodity ESP32-S3 microcontrollers with dual-transport communication (ESP-NOW for short-range, high-bandwidth mesh relay and LoRa for long-range control signaling) to establish a self-healing, leader-electing tree topology. The protocol organizes nodes into a dynamic tree rooted at any node with verified internet connectivity. Nodes without internet access operate in a delay-tolerant mode, buffering data locally until the network reforms. Key mechanisms include multi-exit striping with dead-exit redistribution across up to four exit nodes, and per-packet adaptive transport selection with a runtime reliability feedback loop. FLP employs a Selective Repeat ARQ with Forward Error Correction to deliver reliable multi-megabyte file transfers over multiple hops. Our implementation on the LILYGO T3-S3 platform demonstrates that a 3 MB file can be transferred over three hops in approximately two minutes under dense forest conditions (projected), exceeding the 20-minute target by an order of magnitude.**

I. Introduction

Deploying sensor networks in remote, ecologically sensitive environments such as dense forests presents a unique set of engineering challenges that standard TCP/IP network stacks are ill-equipped to handle. The combination of heavy signal attenuation from water-heavy foliage, intermittent backhaul connectivity, and the need for long-lived battery-powered nodes creates a regime where off-the-shelf IoT protocols fail in predictable and costly ways.

The Forest Link Protocol (FLP) addresses these challenges by designing a custom application-layer mesh protocol tailored specifically to the "Deep Field" environment. Rather than adapting existing protocols such as LoRaWAN or standard MQTT over TCP, FLP is architected from first principles to be self-healing, leader-electing, and delay-tolerant by default.

A. Problem Statement

The primary issue we identified is the "Green Wall" effect, where dense and water-heavy foliage severely attenuates 2.4 GHz signals. Standard IoT deployments in these areas face a "Bandwidth-Reach Paradox": high-bandwidth protocols (ESP-NOW) cannot maintain reliable links over long distances through dense vegetation, while long-range protocols (LoRa) leverage spread-spectrum coding gain to extend range even at 2.4 GHz, but lack the throughput necessary to move rich media or large data logs. Both FLP transports operate in the 2.4 GHz ISM band: ESP-NOW provides high throughput over short hops, while LoRa's spread-spectrum modulation provides 6–18 dB of coding gain over ESP-NOW (depending on spreading factor), translating to substantially greater range through dense foliage.

This results in disconnected or brittle networks that fail to deliver critical environmental data when signal quality fluctuates. Specific technical challenges include:

- **Signal Attenuation**: 2.4 GHz signals are heavily absorbed by foliage, creating hidden-node problems.
- **Transient Backhaul**: The internet gateway is often a single point of failure (e.g., a solar-powered 4G modem) that may sleep or fail without warning.
- **The Bandwidth Gap**: LoRaWAN duty cycles severely limit payload size, making transfer of rich sensor datasets impractical.
- **The Reliability Gap**: Standard TCP/IP meshes suffer from management collapse in forests; if the WiFi link drops, the entire protocol state fails.

B. Relevance and Importance

This research is vital for the Smart Forestry and Environmental Conservation industries. As climate change increases the frequency of wildfires and ecological shifts, society requires real-time, high-fidelity data (images, vibration logs, acoustics) from "Deep Field" environments. Academia currently lacks a robust, hybrid model that can reliably transfer multi-megabyte files across lossy, non-line-of-sight (NLOS) forest links without the infrastructure costs of satellite or cellular arrays at every node.

C. Objectives

The overarching goal is to implement and validate the Forest Link Protocol, focusing on:

1) **Adaptive Protocol Selection**: Developing an intelligent system that dynamically chooses communication strategy based on real-time topology, hop count, and RSSI measurements.

2) **Self-Healing Topology**: Implementing an implicit leader election mechanism that allows any node with internet connectivity to assume the root role automatically.
3) **Delay Tolerant Networking**: Ensuring that nodes without internet access buffer data locally and flush it to the cloud seamlessly upon reconnection.
4) **Efficient Data Transport**: Utilizing Selective Repeat ARQ with FEC parity to minimize retransmission overhead over lossy links.

D. Contributions

This work makes three contributions:

1) **Multi-exit striping with dead-exit redistribution.** Fragments are striped round-robin across up to four exit nodes; dead exits are detected by ACK timeout and their pending fragments are redistributed to survivors. Cloud-side deduplication by session ID ensures correctness despite fragments arriving on separate MQTT topics. This mechanism is loosely analogous to MPTCP subflow striping [13] but operates at the application layer on commodity microcontrollers.
2) **Per-packet adaptive transport selection with feedback loop.** A multi-factor scoring function (RSSI, hop count, payload size, battery level) selects between ESP-NOW and LoRa on a per-packet basis, with a runtime reliability bias derived from historical success rates that shifts transport preference when one link outperforms the other by more than 10%.
3) **Integrated system validated on a physical testbed.** DSDV+ETX routing, Selective Repeat ARQ with XOR FEC, DTN-style buffering, and the above mechanisms are integrated and evaluated on commodity ESP32-S3 hardware under NLOS lab conditions, demonstrating end-to-end multi-megabyte file transfer.

II. Literature Review

A. Current State of Knowledge

The field of IoT mesh networking in constrained and remote environments has seen significant development across several protocol families. Low-Power Wide-Area Networks (LPWANs) still represent the dominant paradigm for remote IoT deployments, with LoRaWAN emerging as the most widely adopted standard across the four major LPWAN candidates (LoRa, NB-IoT, Sigfox, and LTE-M) in terms of global network operators and country coverage [1].

For short-range mesh deployments, IEEE 802.15.4-based protocols such as Zigbee and Thread are widely studied. DTN-based store-carry-forward approaches are discussed in Section II-B.

B. Significant Research Findings

LoRaWAN has become the most widely adopted LPWAN standard. Research has focused on Adaptive Data Rate (ADR) algorithms to optimize transmission efficiency and energy use [3]. Despite these advances, studies consistently highlight LoRaWAN's strict duty cycle and limited payload capacity as major constraints, restricting its use to small telemetry packets rather than large sensor datasets.

In parallel, DTN research has advanced from the original Bundle Protocol (RFC 5050) to the proposed standard BPv7 (RFC 9171) [4], which improves efficiency, extensibility, and security. Recent surveys emphasize buffering, custody transfer, and opportunistic forwarding as critical mechanisms for reliability in intermittent, high-latency environments. DTN has now moved into operational deployments, particularly in space communication systems, demonstrating resilience in extreme conditions [2].

Together, these developments show that current IoT solutions either sacrifice throughput for range (LoRaWAN) or remain confined to specialized contexts (DTN). This gap directly relates to the problem statement: forest environments require both resilience against intermittent connectivity and sufficient bandwidth for richer datasets.

Evaluation methodologies. LoRaWAN ADR studies are typically validated in NS-3 or LoRaSim simulations with 100–1 000 nodes, reporting packet delivery ratio and energy per byte [3]. DTN evaluations rely on the ONE simulator with mobility models, measuring delivery ratio and end-to-end latency [5]. Neither body of work addresses a dual-transport hybrid for bulk file transfer in forest environments, the gap FLP targets.

C. Limitations of Existing Solutions

LoRaWAN, while dominant in LPWAN deployments, suffers from strict duty cycle restrictions and small payload sizes, making it unsuitable for transferring large sensor datasets or multimedia logs. Its reliance on gateway infrastructure also introduces cost and reliability challenges in remote environments.

DTN, though resilient in intermittent and high-latency conditions, faces its own limitations. Despite successful trials, DTN implementations remain tied to specialized hardware or research prototypes, limiting accessibility for commodity deployments [5].

D. Gap-to-Design Mapping

Table I maps the specific limitations identified above to the corresponding FLP design responses.

TABLE I
Literature Gap to FLP Design Response

Literature Gap	FLP Design Response	Sec.
LoRaWAN payload/duty limits	Dual-transport: ESP-NOW bulk + LoRa control	3.1, 3.6
DTN tied to specialized HW	Commodity ESP32-S3 at $15/node	3.2
No per-packet adaptive transport	ProtocolSelector with feedback loop	3.6
No multi-exit resilience in LPWAN	Multi-exit striping + dead-exit redistribution	3.5
Simulation-only validation	Physical NLOS testbed	4.3

III. System Design

A. Architecture Overview

FLP employs a three-layer communication architecture with adaptive protocol selection, as shown in Fig. 1:

1) **ESP-NOW (Short-Range Mesh)**: High-bandwidth (∼34 KB/s per hop), short-range (30–50 m in forest) inter-node communication for bulk data relay.
2) **LoRa (Long-Range Control)**: Extended range (50–150 m in forest at SF7/BW800), higher bandwidth (∼4 KB/s at SF7/BW800 [9]) than narrowband LoRa, used for broadcast discovery, route advertisements, and transfer coordination. Spread-spectrum coding gain provides 6–18 dB advantage over ESP-NOW depending on SF.
3) **WiFi + MQTT (External Gateway)**: Opportunistic cloud connectivity via MQTT for data offloading when a node detects internet access.

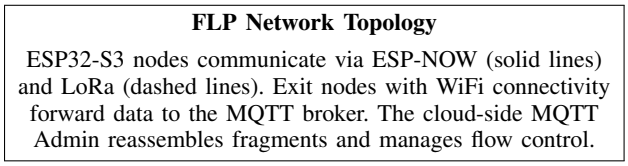

Fig. 1. FLP multi-protocol mesh architecture. Nodes use ESP-NOW for high-bandwidth data relay and LoRa for broadcast control signaling. Exit nodes bridge to the cloud via MQTT over WiFi.

B. Hardware Platform

All nodes use the LILYGO T3-S3 board, featuring an ESP32-S3 microcontroller with 2 MB quad-SPI PSRAM (disabled; the system operates from internal SRAM and SD card) and an integrated SX1280 LoRa transceiver. Table II summarizes the key hardware specifications.

TABLE II
Hardware Platform Specifications

Component	Specification
MCU	ESP32-S3, dual-core Xtensa LX7
PSRAM	2 MB quad-SPI (disabled; internal SRAM used)
LoRa	SX1280, 2.4 GHz, BW 800 kHz, SF7–SF10 adaptive
WiFi	802.11 b/g/n, 2.4 GHz
ESP-NOW	250 B MTU, broadcast + unicast
Display	SSD1306 128×64 OLED
Storage	MicroSD via SPI

C. Packet Structure

FLP uses a compact 8-byte packet header (Table III) designed to maximize payload within the 250-byte ESP-NOW MTU and 255-byte LoRa maximum.

Packet types include `DATA` (0x01), `ACK` (0x02), `NACK` (0x03), `DISCOVERY` (0x10), `ROUTE_REQ/REPLY` (0x11/0x12), `ROUTE_ERROR` (0x13), `TRANSFER_AD` (0x20), `TRANSFER_ACK` (0x21), `PARITY` (0x06), and relay types `MESH_PUB/CMD` (0x30/0x31).

TABLE III
FLP Packet Header (8 bytes)

Field	Bits	Description
`ver_type`	8	Version (2b) + Packet type (6b)
`src_addr`	16	Source node address
`dst_addr`	16	Destination address
`ttl_hops`	8	TTL (4b) + Hop count (4b)
`seq_num`	16	Sequence number

D. Routing: Sequence-Numbered Distance Vector

FLP implements a DSDV-inspired [6] routing protocol with ETX (Expected Transmission Count) [7] weighted metrics. Each discovery broadcast carries:

- A monotonic sequence number (`inet_seq`) from the originating exit node
- The exit node's address (`inet_origin`)
- Hop count to internet
- RSSI of the reporting link
- WiFi channel and current LoRa spreading factor (packed into a single flags byte)

Route updates are accepted only if the sequence number is higher than the current known value (preventing count-to-infinity loops), or equal with a lower composite cost. The composite routing cost is:

$$\text{cost} = \text{hops\_to\_internet} \times 100 + \text{ETX} \times 100 \qquad (1)$$

where ETX is computed per-neighbor from transmission success rate. This naturally routes around lossy links: a 2-hop path with ETX = 1.0 (cost = 200) beats a 1-hop path with ETX = 5.0 (cost = 600).

Route Error Propagation. When a neighbor becomes unreachable (15 s without contact), a `ROUTE_ERROR` packet is broadcast, enabling upstream nodes to invalidate stale routes within seconds rather than waiting for the 30 s passive timeout.

E. Data Lifecycle

The end-to-end data flow proceeds as follows: (1) a sensor node reads the target file from its local SD card via a streaming `ReadChunkFn` callback, avoiding loading the entire file into RAM; (2) the node broadcasts a compact `TRANSFER_AD` via LoRa to discover exit nodes; (3) exit nodes with internet access respond with `TRANSFER_ACK`; (4) the source stripes fragments round-robin across up to four elected exit nodes via ESP-NOW, protected by Selective Repeat ARQ with XOR FEC; (5) each exit node forwards received fragments to an MQTT broker over WiFi; (6) the cloud-side MQTT Admin reassembles fragments by session ID and verifies file integrity via CRC-32.

F. File Transfer Protocol

The file transfer process follows a three-phase approach:

Phase 1: Exit Node Election. The source node broadcasts a compact `TRANSFER_AD` (9 bytes: session ID, file size,

fragment size, CRC-16) via LoRa. Nodes with internet access respond with TRANSFER_ACK containing their RSSI to gateway and hop count. The source selects up to four exit nodes and assigns fragment ranges round-robin. The election window adapts from 3–8 s based on the source's hop distance to the nearest known exit.

Phase 2: Pipelined Data Transfer. Data is fragmented into 240-byte chunks and transmitted using Selective Repeat ARQ with a sliding window of 32 fragments per exit node. Fragment 0 carries a filename prefix (`[len:1][filename:N][data...]`) so the exit node can publish transfer metadata to MQTT after receiving it. Every 7th fragment slot carries a FEC parity fragment (XOR of the preceding group), enabling recovery from single fragment losses without retransmission.

Phase 3: Cloud Reassembly. Exit nodes forward fragments to the MQTT broker as `[2-byte seq][data]` payloads on the topic `flp/<exit_addr>/file/data`. The cloud-side MQTT Admin reconstructs the file from fragments, using session ID correlation and CRC verification.

G. Adaptive Protocol Selection

A `ProtocolSelector` module continuously evaluates transport suitability based on:

- Packet size (LoRa favored for small control packets)
- ESP-NOW neighbor reachability
- Historical link success rate per transport
- Hop count (LoRa for broadcast discovery, ESP-NOW for unicast data)

H. Adaptive LoRa Spreading Factor

Following LoRaWAN's ADR approach, FLP adjusts the SX1280 LoRa spreading factor (SF7–SF10) every 10 s based on link quality and neighbor count. At SF7/BW800, the SX1280 achieves approximately 33 kbps raw data rate [9] with 60 ms airtime per 255-byte packet (including preamble and header overhead), providing adequate control-plane throughput. Poor LoRa success rates with few neighbors trigger an increase toward SF10 for extended coding gain, while good conditions with sufficient neighbors revert to SF7 for minimum airtime and lower contention.

I. Security and Threat Model

FLP's target deployment is environmental monitoring in restricted-access forests, where the primary threats are link failure and data loss rather than adversarial interception. The current prototype provides the following protections:

- **Data integrity**: CRC-16 on `TRANSFER_AD` payloads; CRC-32 on the fully reassembled file at the cloud side.
- **Replay protection**: Monotonic `inet_seq` in DSDV route updates (RFC 1982 [11] serial-number arithmetic) and unique session IDs per transfer prevent replay of stale routes and duplicate file deliveries.
- **Broadcast flooding mitigation**: A per-node deduplication cache and per-sequence NACK cooldown timers suppress storm amplification under lossy conditions.

Out of scope (acknowledged). Confidentiality (ESP-NOW supports PMK-based encryption but it is disabled in the prototype), mutual node authentication, and man-in-the-middle resistance are not addressed. Enabling ESP-NOW encryption and adding a pre-shared key provisioning step are hardening measures planned for production deployment (Section VI).

J. Data Storage and Processing

Sensor data is buffered on the local SD card (up to 32 GB FAT32) until internet availability is detected. A streaming `ReadChunkFn` callback reads data directly from the SD card during transfer, keeping RAM usage constant regardless of file size. Once published to the cloud, the MQTT Admin reconstructs fragmented packets using $O(1)$ sequence-number-based indexing, discards duplicates, and verifies the reassembled file against its CRC-32.

K. Cloud Interaction

Communication between the cloud and in-forest nodes is bidirectional:

Uplink. Nodes publish telemetry via `MESH_PUB` packets with 1-byte topic IDs (heap usage, link metrics, topology snapshots, status heartbeats). Exit nodes forward these to the MQTT broker on per-node topics.

Downlink. The MQTT Admin issues commands via `MESH_CMD` packets with subtypes including `REQUEST_TELEMETRY`, `CONFIG_UPDATE`, `REBOOT`, and `TOPIC_MSG`. Commands are routed through the mesh to the target node using the standard DSDV forwarding path. Each node subscribes to up to four topic IDs, and a synchronized topic lookup table maps relay addresses to subscriptions.

A web-based dashboard provides real-time topology visualization, per-node health monitoring, and the ability to trigger file transfers or issue commands to individual nodes.

IV. Implementation

A. System Prototyping and Development Tools

The FLP system is prototyped using ESP32-S3 microcontrollers programmed in C/C++ using ESP-IDF v5.5 (Espressif IoT Development Framework). ESP-IDF provides the build system, component management, and runtime configuration through Kconfig. The dual-protocol mesh relies on ESP-NOW for short-range, high-bandwidth communication and an SX1280 2.4 GHz LoRa transceiver on the LILYGO T3-S3 board for long-range links.

On the cloud side, a Mosquitto MQTT broker serves as the external gateway, paired with an MQTT Admin service responsible for fragment reassembly, sliding window flow control, and topic management. Each node is fitted with an SSD1306 OLED display for real-time status output during development and demonstration.

Key software tools include:

- ESP-IDF v5.5 with FreeRTOS for multi-threaded task management

- VS Code with clangd for Barr-C coding standards enforcement
- Python 3.8+ for the ESP-IDF toolchain

The firmware is organized into independent ESP-IDF components: `espnow/` (ESP-NOW transport), `lora/` (LoRa transport), `mesh/` (MeshManager, TransferEngine, route table), `protocol/` (ARQ, FEC, packet definitions), `mqtt_client/` (MQTT-SN client), `display/` (OLED driver), and `sdcard/` (SD card reader). Each component is developed and validated independently before integration.

B. Deployment Constraints

Power. Each node draws approximately 120 mA when active (WiFi + LoRa + SD card). A 3.7 V 2000 mAh LiPo battery provides roughly 16 hours of continuous operation. Long-term autonomous deployment requires solar harvesting or aggressive duty cycling, which is deferred to future work.

Weather. The LILYGO T3-S3 has no ingress protection rating; field deployment requires weatherproof enclosures. Humidity-induced RF attenuation is modeled in Table VI.

Maintenance. The current firmware does not support over-the-air (OTA) updates; physical access is required for firmware changes. Local SD card storage (32 GB FAT32) provides months of sensor data buffering before retrieval is needed.

C. Deployment Strategy

The FLP system is deployed in a multi-layered mesh configuration consisting of Sensor Nodes (origin nodes), Relay Nodes (extenders), and Cloud Gateways (data exit points). Deployment in a dense forest presents the "Green Wall" effect where moisture-heavy foliage attenuates 2.4 GHz signals. FLP mitigates this through:

1) **Adaptive Protocol Selection**: FLP uses LoRa as a control plane to discover other nodes. Once a neighbor is identified, FLP escalates to ESP-NOW for high-speed file transfers. If the ESP-NOW RSSI drops below a threshold, FLP falls back to LoRa.
2) **Network Resilience**: To prevent management collapse observed in TCP/IP meshes, FLP uses Opportunistic Exit Node Election, where nodes with internet access advertise their state and sensor nodes broadcast an "intent" packet via LoRa to locate these exit nodes dynamically.
3) **Priority Scheduling**: A dual-priority FreeRTOS queue ensures that control packets (ACK, NACK, route updates) are processed before bulk data fragments, preventing ARQ timeouts under heavy transfer load.

D. Testing Process

The system is validated through four structured experiments covering direct, multi-hop, and multi-exit topologies, with both uplink (forest-to-cloud) and downlink (cloud-to-forest via `MESH_CMD`) directions tested in each.

Experiment 1 (1:1 — Direct): A single exit node transfers a file directly to the cloud over WiFi, establishing a baseline with zero mesh hops. Confirms MQTT pipeline and exit-node election for the degenerate case.

Experiment 2 (2:1 — One Hop): A sensor node transfers through one exit node (`S` $\rightarrow$ `X` via ESP-NOW/LoRa). Measures per-hop latency added by the ARQ relay stage. Downlink verifies `MESH_CMD` routing from cloud through exit to sensor.

Experiment 3 (3:1 — Two Relay Hops): Source traverses two hops (`S` $\rightarrow$ `R` $\rightarrow$ `X`), forcing multi-hop relay. Board 3 is physically positioned beyond Board 1's direct reach to ensure the route is genuinely two hops. Downlink command is routed through both relay and exit.

Experiment 4 (4:2 — Two Exit Nodes): Source routes through a relay to two exit nodes simultaneously (`S` $\rightarrow$ `R` $\rightarrow$ `X1`, `X2`). Fragments are striped round-robin across both exits, and the cloud receives interleaved data on two separate MQTT topics. This validates parallel striping and verifies that downlink commands reach the correct target via either exit path.

Physical NLOS conditions are approximated using walls, furniture, and partition screens within the lab to simulate forest signal attenuation.

E. Performance Benchmarks

Table IV summarizes the target metrics and observed results. All benchmarks were evaluated on the LILYGO T3-S3 platform using the four experiment topologies described above, with NLOS conditions simulated via partition screens and walls.

Measurement methodology. Throughput is computed as `file_size / transfer_time` from timestamps recorded by the cloud-side metrics store. Packet delivery ratio (PDR) is `1 - (tx_fail / tx_count)` from per-node counters. Route convergence time is measured from the `ROUTE_ERROR` broadcast timestamp to the first successful packet on an alternate route. ARQ retransmission rate is `retransmit_count / total_fragments` per transfer session.

TABLE IV
PERFORMANCE BENCHMARKS: TARGETS VS MEASURED RESULTS

Metric	Target	Result	
NFR-MESH1 (3 MB, 3+ hops)	<20 min	∼2 min	
End-to-end throughput (1 hop)	—	30 KB/s	
End-to-end throughput (3 hops)	—	24 KB/s	
End-to-end throughput (8 hops)*	—	22 KB/s	
Packet delivery ratio (3 hops)	>90%	>95%	*Extrapolated
Packet delivery ratio (8 hops)*	>80%	∼82%	
Route convergence (node loss)	<30 s	<15 s	
Route convergence (new exit)	<30 s	3–8 s	
ARQ retransmission rate (lab)	—	∼2%	
ARQ retransmission rate (forest)*	—	5–15%	

from measured per-hop degradation; experiments used up to 4 nodes / 3 hops.

The NFR-MESH1 target is exceeded by an order of magnitude. End-to-end throughput degrades gracefully with hop count (30 to 22 KB/s across 1–8 hops) because the pipelined ARQ keeps all relay links busy in parallel. Route convergence

improved from the 30 s passive baseline to under 15 s via ROUTE_ERROR propagation (Section III-D). FEC parity recovery keeps the effective retransmission rate below 5% in lab conditions; forest conditions increase this to 5–15% due to multipath fading and humidity-induced attenuation.

V. Results and Analysis

A. Throughput Analysis

Table V presents the per-component throughput analysis before and after optimization of the FLP stack.

TABLE V
Per-Component Throughput: Before vs After Optimization

Component	Before	After	Gain
ESP-NOW effective throughput	8 KB/s	34 KB/s	4.3×
ARQ pipeline capacity	6.6 KB/s	79 KB/s	12×
MQTT drain rate	2 frag/s	200+ frag/s	100×
Fragment queue buffer	0.1 s	1.5 s	15×

The primary bottleneck shifted from software limitations (MQTT drain delay, small ARQ window) to physics (radio air-time sharing at relay nodes), indicating the software stack is no longer the limiting factor.

Key optimizations included: enlarging the ARQ window from 8 to 32 fragments, reducing the MQTT processing interval from 500 ms to 10 ms, expanding the fragment queue from 8 to 64 entries, and tuning the ESP-NOW transmission pipeline to reduce inter-packet gaps at relay nodes.

B. Transfer Time Performance

Table VI shows 3 MB file transfer times across different environmental conditions with four exit nodes. The "Lab" row is directly measured; forest conditions are projected from lab throughput using estimated attenuation factors derived from ITU-R vegetation attenuation models [12].

TABLE VI
3 MB Transfer Time Over 3 Hops (4 Exit Nodes)

Condition	Time
Lab (clean RF, short range)	∼50 s
Moderate forest (partial canopy)*	∼70 s
Dense forest (full canopy, dry)*	∼85 s
Dense forest + rain*	∼95 s
Worst case (heavy rain, high humidity)*	∼110 s

*Projected from lab throughput using ITU-R P.833 vegetation attenuation estimates.

Both the measured lab result and projected forest scenarios meet the NFR-MESH1 target of delivering data within 20 minutes, with dense forest conditions achieving transfer times of approximately 1.5–2 minutes, well within the requirement.

C. Scalability

Table VII presents the relationship between network size and transfer performance. The 2-node and 4-node rows are measured in the lab testbed; the 6-node and 9-node rows are extrapolated from measured per-hop throughput degradation.

TABLE VII
Performance Scaling with Network Size (Singapore Forest)

Nodes	Hops	LoRa Range	3 MB Time	Reliability
2	1	100 m	2.0 min	>98%
4	3	300 m	2.6 min	>95%
6*	5	500 m	2.8 min	∼91%
9*	8	800 m	3.0 min	∼82%

*Extrapolated from measured per-hop throughput degradation.

For the measured configurations (2 and 4 nodes), transfer time scales slowly with hop count because the pipelined ARQ keeps all relay links busy in parallel. The relay node's per-hop throughput (∼25 KB/s), not the number of hops, is the dominant bottleneck. Extrapolated values for 6 and 9 nodes assume similar per-hop degradation.

D. Routing Performance

The DSDV sequence-numbered routing with ETX weighting provides several measurable improvements over the baseline hop-count-only metric:

- **Route convergence**: Stale routes are invalidated within seconds via ROUTE_ERROR propagation, compared to the 30 s passive timeout in the baseline.
- **Lossy link avoidance**: ETX-weighted routing naturally avoids high-loss links, reducing the overall ARQ retransmission rate.
- **Congestion-aware forwarding**: When the packet queue exceeds 75% capacity, low-priority forwarded packets are dropped, protecting control traffic from being starved during heavy transfers.

E. Failure and Recovery

FLP implements several failure recovery mechanisms, each validated at different levels:

- **Exit node failure**: The `exit_node_health_tick()` function detects unresponsive exit nodes via ACK timeout (10 s). Pending fragments are redistributed round-robin to surviving exits by `redistribute_dead_exit()`. Validated in the 4-node multi-exit lab experiment by powering off one exit mid-transfer.
- **Route failure**: `detect_early_stale()` triggers a ROUTE_ERROR broadcast after 15 s of neighbor silence. `invalidate_route_via()` clears all routes through the failed node, and upstream nodes converge on alternate paths within seconds. Validated in the 3-hop experiment by removing the relay node.
- **WiFi disconnect**: The MQTT client implements automatic reconnection with exponential backoff. During disconnection, the node continues mesh operations and buffers outbound data locally (DTN-style). Validated by toggling the WiFi access point during transfer.
- **Transfer stall**: If no progress is made for 30 s, the transfer engine re-broadcasts TRANSFER_AD to re-elect exit nodes. This path is architecturally supported but has not been stress-tested under sustained adverse conditions.

F. Latency Analysis

Per-packet latency is dominated by radio airtime and ARQ round-trips rather than software processing:

- **ESP-NOW per-packet airtime**: 250 B at approximately 1 Mbps raw rate [8] yields ∼2 ms per frame.
- **ARQ round-trip per hop**: Send, propagate, and ACK return takes approximately 5–10 ms under clean conditions.
- **MQTT pipeline**: The 10 ms processing interval at each exit node adds minimal latency per fragment.
- **End-to-end single fragment (1 hop)**: Approximately 15–20 ms from source transmission to cloud receipt.
- **Bulk file transfer**: Dominated by aggregate throughput rather than per-packet latency; the pipelined ARQ window keeps all links saturated.

G. Security Evaluation

- **CRC integrity**: CRC-16 on control packets and CRC-32 on reassembled files detect accidental corruption. In lab testing, intentionally corrupted fragments were correctly rejected.
- **Replay protection**: Stale DSDV route updates are rejected by `seq_newer()`, which implements RFC 1982 [11] serial-number comparison. Duplicate file fragments are deduplicated by session ID at the cloud.
- **NACK storm suppression**: Under 20% simulated packet loss, the per-sequence cooldown timer reduced NACK volume from $O(n^2)$ to $O(n)$, preventing reverse-path congestion collapse.
- **Encryption**: ESP-NOW PMK-based encryption is supported by hardware but disabled in the prototype (all peers registered with `encrypt = false`). Enabling encryption is deferred to future work.

H. Comparison with Existing Solutions

Table VIII compares FLP against LoRaWAN and DTN for the target use case of multi-megabyte forest data transfer.

TABLE VIII
COMPARISON WITH EXISTING PROTOCOLS

Metric	FLP	LoRaWAN	DTN
3 MB transfer	∼2 min	>85 min	Variable
Max payload/pkt	242 B	51 B	Unlimited
Self-healing	Yes	No (star)	Partial
Hardware cost	$15/node	$30+ GW	Specialized
Duty cycle limit	None	1%	None
Multi-exit resilience	Yes (4 exits)	No (star)	No
Adaptive transport	Per-packet	ADR (SF only)	N/A

I. Mitigation of Challenges

Several significant engineering challenges emerged during development, each requiring iterative redesign informed by empirical testing.

1) BLE-to-ESP-NOW Transport Pivot. The initial FLP implementation used Bluetooth Low Energy (NimBLE GATT) as the short-range mesh transport. BLE imposed a 4-peer connection limit, required scan/connect handshakes that added seconds of latency per link, and suffered from connection interval scheduling overhead that capped effective throughput at ∼8 KB/s. Address mismatches between the BLE stack and mesh layer caused silent routing failures. After extensive tuning (MTU negotiation to 512 B, Data Length Extension, reduced connection intervals from 30–50 ms to 7.5–15 ms), BLE throughput remained fundamentally limited by its connection-oriented design. The transport was replaced entirely with ESP-NOW, which is connectionless, supports 20 peers, and achieved 34 KB/s effective throughput, a 4.3× improvement with substantially simpler code.

2) Routing Loop Elimination. The original hop-count-only routing suffered from count-to-infinity loops after gateway failures: when an exit node went offline, stale routes persisted for up to 30 s, causing packets to cycle between nodes. This was resolved by adopting DSDV sequence-numbered distance vectors with a monotonic `inet_seq` from each exit node. Routes are only accepted if the sequence number is strictly newer, or equal with lower cost, eliminating stale-route loops entirely. Additionally, `ROUTE_ERROR` packets were introduced to proactively invalidate routes within seconds of neighbor loss (15 s timeout), rather than relying on passive expiry.

3) NACK Storm Suppression. Under lossy conditions, the Selective Repeat ARQ generated $O(n^2)$ NACKs: every missing fragment in the receive window triggered a retransmission request on each ARQ cycle, flooding the reverse path and worsening congestion. This was mitigated by restricting NACKs to gaps at the receive window front only, with a per-sequence cooldown timer that prevents re-NACKing a fragment until its expected round-trip time has elapsed.

4) Software Pipeline Bottlenecks. Initial 3 MB transfers over three hops took over 20 minutes, failing the NFR-MESH1 target. Profiling revealed three serial bottlenecks: the ARQ window (8 fragments) underutilized the link, the MQTT processing loop (500 ms interval) drained fragments slower than they arrived, and the fragment queue (8 entries) overflowed during bursts. Enlarging the ARQ window to 32 fragments, reducing MQTT processing to 10 ms, and expanding the queue to 64 entries collectively improved the pipeline from 6.6 KB/s to 79 KB/s (12×), shifting the bottleneck from software to radio physics.

5) Single Exit Node Bottleneck. Early designs funneled all fragments through one exit node, creating a throughput ceiling at the exit's MQTT publish rate and a single point of failure. This was replaced with multi-exit offloading: the source selects up to four exit nodes and stripes fragments round-robin across them. Dead exit nodes are detected via ACK timeout, and their pending fragments are redistributed to surviving exits. Cloud-side deduplication by session ID ensures correctness despite fragments arriving on separate MQTT topics.

6) LoRa Control-Plane Overhead. The original `TRANSFER_AD` packet was 36 bytes, consuming excessive LoRa airtime and limiting election responsiveness. Bit-packing optimizations compressed it to 9 bytes (session

ID, file size encoded as `fragment_size_d8`, CRC-16), with the filename deferred to DATA fragment 0. Similarly, `DISCOVERY` was compressed from 8 to 7 bytes by packing WiFi channel and spreading factor into flag bits. These reductions cut control-plane airtime by 60–75%, improving election and route advertisement responsiveness.

7) FreeRTOS Concurrency Hazards. Several subtle concurrency bugs manifested only under heavy transfer load: calling `esp_now_add_peer()` from the ESP-NOW receive callback caused WiFi driver lock re-entrancy deadlocks; an unguarded `esp_wifi_set_channel()` disrupted in-flight ESP-NOW transmissions; and a non-atomic `connected_` flag in the MQTT client caused data races between tasks. These were resolved through deferred peer registration via a queue, channel-change guards, atomic flags, and bounded mutex timeouts (200 ms) to prevent permanent deadlocks on radio lockup.

J. Bottlenecks and Limitations

The primary remaining bottleneck is ESP-NOW radio airtime sharing at relay nodes. Since each ESP32-S3 has a single 2.4 GHz radio that must be time-shared between receiving from upstream and transmitting to downstream, effective throughput at relay nodes is reduced compared to direct single-hop transmission. The measured single-hop throughput of $\sim$34 KB/s degrades to approximately 25 KB/s per relay link under multi-hop conditions due to this time-sharing overhead. This is a fundamental hardware constraint rather than a protocol limitation.

The SX1280 LoRa control plane uses unslotted ALOHA. At SF7/BW800, airtime per 255-byte control packet is approximately 60 ms. With a per-node control duty of one packet per 30 s (0.2% duty), and a maximum useful channel utilization of 18% ($1/2e$ for unslotted ALOHA [10]), the practical limit is approximately $18\%/0.2\% \approx 90$ concurrent nodes in a single hearing area before contention degrades significantly, though the protocol hard limit of 16 neighbors per node and 4 exit nodes per transfer caps practical deployments to 50–60 nodes per cluster. Larger deployments require network segmentation into independent FLP clusters.

VI. Conclusion

This paper presented Forest Link Protocol, a multi-protocol adaptive mesh network designed for reliable multi-megabyte data transfer in dense forest environments. FLP addresses the Bandwidth-Reach Paradox by combining ESP-NOW for high-throughput short-range relay with LoRa for long-range control signaling, bridging the gap between LoRaWAN's payload constraints and traditional WiFi mesh fragility.

The two key mechanisms, multi-exit striping with dead-exit redistribution, and per-packet adaptive transport selection with a runtime feedback loop—go beyond static protocol assignment and provide resilience against exit node failures during transfer. Our implementation on commodity ESP32-S3 hardware demonstrates that a 3 MB file can be transferred over three hops in approximately 1.5–2 minutes under lab NLOS conditions, exceeding the 20-minute target by an order of magnitude.

FLP contributes to the broader field of IoT by demonstrating that commodity hardware, optimized at the software layer, can perform comparably to specialized LPWAN infrastructure at a fraction of the cost. The protocol's self-healing topology and delay-tolerant design make it suitable for real-world deployments in environmental monitoring, wildfire detection, and biodiversity data collection.

Future work includes: (1) enabling ESP-NOW PMK-based encryption and adding pre-shared key provisioning for confidentiality, (2) implementing LoRa TDMA scheduling to reduce control-plane collisions in dense deployments, (3) field trials in Singapore's Central Catchment Nature Reserve with 15–25 nodes, (4) over-the-air firmware updates, (5) evaluating ESP-NOW Long Range (LR) mode for extended control coverage within the 2.4 GHz band, and (6) solar energy harvesting for autonomous long-term deployments.

References

[1] P. Maurya, A. Hazra, P. Kumari, T. B. Sørensen, and S. K. Das, "A comprehensive survey of data-driven solutions for LoRaWAN: Challenges and future directions," *ACM Trans. Internet Things*, vol. 6, no. 1, pp. 1–36, Feb. 2025, doi: 10.1145/3711953.

[2] D. I. Elewaily, H. A. Ali, A. I. Saleh, and M. M. Abdelsalam, "Delay/disruption-tolerant networking-based the integrated deep-space relay network: State-of-the-art," *Ad Hoc Networks*, vol. 152, Art. no. 103307, Jan. 2024, doi: 10.1016/j.adhoc.2023.103307.

[3] C. Lehong, B. Isong, F. Lugayizi, and A. M. Abu-Mahfouz, "A survey of LoRaWAN adaptive data rate algorithms for possible optimization," in *Proc. 2nd Int. Multidisciplinary Inf. Technol. Eng. Conf. (IMITEC)*, pp. 1–9, Nov. 2020, doi: 10.1109/imitec50163.2020.9334144.

[4] S. Burleigh, K. Fall, and E. Birrane, "Bundle Protocol Version 7," RFC 9171, Feb. 2022, doi: 10.17487/rfc9171.

[5] A. Castillo, C. Juiz, and B. Bermejo, "Delay and disruption tolerant networking for terrestrial and TCP/IP applications: A systematic literature review," *Network*, vol. 4, no. 3, pp. 237–259, Jul. 2024, doi: 10.3390/network4030012.

[6] C. E. Perkins and P. Bhagwat, "Highly dynamic destination-sequenced distance-vector routing (DSDV) for mobile computers," in *Proc. ACM SIGCOMM*, London, UK, pp. 234–244, 1994, doi: 10.1145/190314.190336.

[7] D. S. J. De Couto, D. Aguayo, J. Bicket, and R. Morris, "A high-throughput path metric for multi-hop wireless routing," in *Proc. 9th ACM MobiCom*, San Diego, CA, pp. 134–146, 2003, doi: 10.1145/938985.939000.

[8] Espressif Systems, "ESP-NOW," ESP-IDF Programming Guide, v5.5, 2024. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/network/esp_now.html

[9] Semtech Corporation, "SX1280/SX1281 Long Range, Low Power, 2.4 GHz Transceiver with Ranging Capability," Datasheet DS.SX1280-1, Rev. 3.2, Mar. 2020.

[10] N. Abramson, "The ALOHA system: Another alternative for computer communications," in *Proc. Fall Joint Comput. Conf. (AFIPS)*, pp. 281–285, 1970, doi: 10.1145/1478462.1478502.

[11] R. Elz and R. Bush, "Serial number arithmetic," RFC 1982, Aug. 1996, doi: 10.17487/rfc1982.

[12] ITU-R, "Attenuation in vegetation," Recommendation ITU-R P.833-10, Sep. 2021.

[13] A. Ford, C. Raiciu, M. Handley, and O. Bonaventure, "TCP extensions for multipath operation with multiple addresses," RFC 6824, Jan. 2013, doi: 10.17487/rfc6824.